

ファイル操作の応用

コマンドでテキストファイルを開覧するにはどうすればよいでしょうか。Windows であればメモ帳など多数のエディタが存在しています。

Linux では cat コマンドを使用して確認できました。

```
[hal@motobe ~]$ cat /etc/passwd
```

以下のコマンド群を用いて効率的なファイル操作をマスターして下さい。(小ネタが多いので、ガンガン進みましょう！)

(1) less ←P130

less コマンドはファイルが長い場合でも先頭から表示させます。テキストファイルの内容をページ単位で表示します。

```
[hal@motobe ~]$ less /etc/passwd
```

※終了するには、「q」を押す

※似たような機能で性能の低い(?)more コマンド(P130)も一緒に確認しておいて下さい。

(2) head ←P134

head コマンドは、ファイルの先頭部分を表示します。オプションをつけずに実行すると先頭から 10 行表示されます。

```
[hal@motobe ~]$ head /etc/passwd
```

もしくは

```
[hal@motobe ~]$ cat /etc/passwd | head
```

※もし 3 行だけを表示させたい場合は、以下のオプションを付け、実行します。

```
[hal@motobe ~]$ head -3 /etc/passwd
```

もしくは

```
[hal@motobe ~]$ cat /etc/passwd | head -3
```

←「| (パイプ)」は後述

(3) tail ←P136

tail コマンドは**ファイルの終わり部分を表示**します。オプションをつけずに実行すると終わりから 10 行表示されます。

```
[hal@motobe ~]$ tail /etc/passwd
```

もしくは

```
[hal@motobe ~]$ cat /etc/passwd | tail ←「| (パイプ)」は後述
```

※もし終わりから 3 行だけを表示させたい場合は、以下のオプションを付けて実行します。

```
[hal@motobe ~]$ tail -3 /etc/passwd
```

もしくは

```
[hal@motobe ~]$ cat /etc/passwd | tail -3
```

tail はファイルの終わり部分を表示するコマンドですが、ファイルによっては、その終わり部分が随時変わることがあります。 ←ログファイル等

tail に-f オプションをつけることにより、**変更をリアルタイムでモニタすることが可能**になります。

```
[hal@motobe ~]$ sudo tail -f /var/log/messages
```

(4) sort ←P138

sort コマンドは、**テキストファイルの行を並び替え**ます。

```
[hal@motobe ~]$cat test1.txt ←各自で用意しておくこと
```

```
ghi qjk
def arf
abc frk
```

①各行の 1 文字目を辞書式(a→d→g) で並び替えます。

```
[hal@motobe ~]$sort test1.txt
```

```
abc frk
def arf
ghi qjk
```

②逆順に並び替えます(g→d→a)。

```
[hal@motobe ~]$ sort -r test1.txt
```

```
ghi qjk
def arf
abc frk
```

③2 列目のデータを並び替えます。

```
[hal@motobe ~]$ sort -k 2 test1.txt
```

```
def arf
abc frk
ghi qjk
```

※いろいろと試してくださいね。

(5) **uniq** ←P140

uniq コマンドは**テキストファイルの行の重複**を消去します。連続している同じ内容の行を 1 行にまとめることができます。

```
[hal@motobe ~]$ cat test2.txt ←各自で用意しておくこと
```

```
aa
dd
kk
cc
cc
ee
```

①連続した「cc」列を 1 行にまとめます。

```
[hal@motobe ~]$ uniq test2.txt
```

```
aa
dd
kk
cc
ee
```

「cc」が 1 行にまとめられましたね。

②重複な行を表示します。

```
[hal@motobe ~]$ uniq -d test2.txt
```

```
cc
```

(6)リダイレクトについて(まあまあ重要)

←P172～

Linux コマンドの多くは、「標準入力」という入口からデータを取り込み、処理の結果を「標準出力」という出口へ送り出すように作られています。

Linux では通常、コマンドを実行すると **キーボードからの入力が受け付け**られ、その処理の結果がディスプレイに表示されますがこれは標準入力キーボードがそして **標準出力にディスプレイが最初から割り当てられている**からです。

「**リダイレクト**」と呼ばれる機能を利用するとこの割り当てを自由に変更できます。つまり、コマンドに対して、キーボード入力の代わりにファイルからデータを出力させたりすることができるのです。

【標準出力の場合】

```
[hal@motobe ~]$ date
```

```
2025 年  6 月 13 日 金曜日 10:48:54 JST
```

【リダイレクトの場合】

```
[hal@motobe ~]$ date > test3.txt
```

```
[hal@motobe ~]$ cat test3.txt
```

```
2025 年  6 月 13 日 金曜日 10:49:45 JST
```

この例の場合、「date」の実行結果は、ディスプレイに出力される代わりに「**test3**」ファイルに出力されます。これは、「**>**」記号の使用によって標準出力が「test3」ファイルに切り替えられたからとなります。

c at コマンドで「test3」ファイルを参照すると時刻が表示されますね。

(7)追加のリダイレクトについて

←P172～

(復習)【リダイレクトの場合】

```
[hal@motobe ~]$ date > test3.txt
```

```
[hal@motobe ~]$ cat test3.txt
```

2025 年 6 月 13 日 金曜日 10:50:18 JST

(例えば 2 日後)以下のように再度実行すると、以前の内容が上書きされます。

```
[hal@motobe ~]$ date > test3.txt
```

```
[hal@motobe ~]$ cat test3.txt
```

2025 年 6 月 15 日 日曜日 10:50:18 JST

※アクセス時間が更新されましたね。

● 2025 年 6 月 13 日の内容も残しつつ、2025 年 6 月 15 日を追加したい場合は、追加のリダイレクトを使用します。

【追加のリダイレクトの場合】

```
[hal@motobe ~]$ date >> test3.txt
```

```
[hal@motobe ~]$ cat test3.txt
```

2025 年 6 月 13 日 金曜日 10:50:18 JST

2025 年 6 月 15 日 日曜日 10:51:27 JST

今度は追加されました。利用目的で使い分けましょう！！

(8)標準エラー出力について

←P172～

Linux を操作していると、様々なエラーが発生します。エラーをユーザーに伝える場合、主にモニタに表示します。これを**標準エラー出力**といいます。標準エラー出力をファイルへリダイレクトさせることもできます。

```
[hal@motobe ~]$ ls -l test4.txt
```

ls: 'test4.txt' にアクセスできません: そのようなファイルやディレクトリはありません

※まあ、ファイルが無いので当たり前ですね。

```
[hal@motobe ~]$ ls -l test4.txt > error.txt
```

ls: 'test4.txt' にアクセスできません: そのようなファイルやディレクトリはありません

※エラー内容をリダイレクトすると・・・失敗しました。

```
[hal@motobe ~]$ ls
```

FTP_Test.txt test.txt ダウンロードデスクトップビデオ画像

error.txt test3.txt テンプレートドキュメント音楽公開

※「error.txt」ファイルは出来ています

```
[hal@motobe ~]$ cat error.txt
```

※なにも書き込まれていません

```
[hal@motobe ~]$ ls -l test4.txt > error.txt 2>&1
```

※ 2 は、標準エラー出力を示しています。標準出力は 1 になります。

※ もし標準出力と標準エラー出力を混ぜて出力したい場合は、上記のように入力します。

```
[hal@motobe ~]$ cat error.txt
```

ls: 'test4.txt' にアクセスできません: そのようなファイルやディレクトリはありません

※エラー内容をリダイレクト出来ました。

(9)パイプについて ←P173

例えば以下のコマンドを実行します。

```
[hal@motobe ~]$ ls -l /usr/bin/
```

画面にファイルの一覧が表示されます。しかし/usr/bin ディレクトリには数多くのコマンドが配置されているため、画面にすべてを表示することができません。ここで以下のように実行します。

```
[hal@motobe ~]$ ls -l /usr/bin/ | less
```

ここで使われている【 | 】がパイプと呼びます。パイプの役割は標準出力と標準入力を「つなげる」ことにあります。

今回の場合、ls コマンドの標準出力から出た**ファイル一覧と、less コマンドの標準入力をつなげます**。これによりページングすることができます。先頭行から表示させ、最終行まで順々に表示させることが可能となります。

【q】を入力することでページャ起動を終了させることができます。

※これからもパイプを使うことがありますので、覚えておきましょう！！

(10) grep ←P142

grep コマンドは、ファイルの中から**データを検索**します。「 ~ | grep 」することで、標準入力から入ったデータに対して検索を行いこともできます。検索する場合、**正規表現が用いられる**場合もあります。

①/etc/passwd ファイルの hal 行を検索

```
[hal@motobe ~]$ grep hal /etc/passwd
```

```
halt:x:7:0:halt:/sbin:/sbin/halt
```

```
hal:x:1000:1000:hal:/home/hal:/bin/bash
```

②ps コマンドの表示結果から、「httpd」文字列を検索

```
[hal@motobe ~]$ ps aux | grep http
```

```
hal          6288  0.0  0.1 221676  2304 pts/0    S+   21:52   0:00 grep --color=auto http
```

③/usr/bin/配下の a で始まるファイル名を検索

```
[hal@motobe ~]$ ls /usr/bin/ | grep ^a ←^は正規表現 : P144
```

④/usr/bin/配下の b で終わるファイル名を検索

```
[hal@motobe ~]$ ls /usr/bin/ | grep b$ ←$は正規表現 : P144
```

(11)正規表現 ←P144

【正規表現の主なメタキャラクタ】

- ・ → 任意の 1 文字にマッチする
- * → 直前にある文字の 0 回以上の繰り返しにマッチする
- ^ → 行頭にマッチ
- \$ → 行末にマッチする
- [] → []内の任意の 1 文字にマッチ
- ¥ → 直後に続くメタキャラクタの意味を解除する
- + → 直前にある文字の 1 回以上の繰り返し
- ? → 直前にある文字の 0 回以上もしくは 1 回の繰り返し
- | → | の左右 1 文字の記述いずれかにマッチする

【正規表現の例】

a.c → 「abc」「a1c」などにマッチ**.*** → 0 文字以上の文字列にマッチ**[A-Z]** → 大文字のアルファベット 1 文字にマッチ**[0-9][0-9]** → 2 桁の数字にマッチ**[a-zA-Z]+** → アルファベットのみの文字列にマッチ**^\$** → 空行（改行のみの行）にマッチ**.*~~¥~~.txt** → 「.txt」で終わる文字列にマッチ(12) **tr** ←教科書に無い**tr** コマンドは標準入力からデータを文字列で置き換えることができます。

【書式】

tr [オプション] 文字列 1 文字列 2

※オプション

-c, 文字列 1 で指定した文字以外を文字列 2 に置き換える

-d, 文字列 1 で指定した文字を削除

【実行例】

[hal@motobe ~]\$ cat test4.txt ←test4 は各自で用意しておくことabc
def
adf

①a を A、f を F へ置き換えます。

[hal@motobe ~]\$ cat test4.txt | tr af AF**A**bc
de**F**
Ad**F**

②af を削除する。

[hal@motobe ~]\$ cat test4.txt | tr -d afbc
de
d

(13)diff ←重要！P147

diff コマンドは、**2つのファイルを行ごと比較**します。

【書式】

diff [オプション] ファイル 1 ファイル 2

オプション

- i,ファイル内容の大文字、小文字を無視する
- b,空白数の差を無視する
- a,すべてテキストとして処理する
- q,ファイルが異なるかどうかだけを報告する
- r,見つかった下位ディレクトリを再帰的に比較する

【実行例】

```
[hal@motobe ~]$ cat test5.txt ←あらかじめ作成
```

```
abc
123
hal.com
```

```
[hal@motobe ~]$ cat test6.txt ←あらかじめ作成
```

```
abc
123
motobe.com
```

①test5.txt と test6.txt を比較する

```
[hal@motobe ~]$ diff test5.txt test6.txt
```

```
3c3
< hal.com
---
> motobe.com
```

※「3c3」とは、test5.txt の 3 行目と test6.txt の 3 行目を change するということです。

②test5 と test6 の内容が異なるどうかを表示する

```
[hal@motobe ~]$ diff -p test5.txt test6.txt
```

```
*** test5.txt      2023-06-17 22:04:05.278260285 +0900
--- test6.txt      2023-06-17 22:04:36.311159773 +0900
```

*** 1,3 ****

abc

123

! hal.com

--- 1,3 ----

abc

123

! motobe.com

```
[hal@motobe ~]$ cp test5.txt test7.txt
```

```
[hal@motobe ~]$ diff -p test5.txt test7.txt
```

※test7.txt は test5.txt のコピーなので内容は同じ。なので何も表示されない。

以上！

他にも大切なコマンドは沢山あります。

秋から始める Linux サーバ構築でも基本コマンドは学習して行きます。